

Project and Interface Handbook

A complete explanation of the product story, role-based interfaces, information access, workflows, APIs, and current backend architecture.

PLATFORM	CURRENT IMPLEMENTATION
AI-powered cold-chain monitoring for temperature-sensitive shipments	Four role-specific web experiences connected to a Python API and an in-memory data store
Supported organizations	Hospitals, pharmacies, laboratories, supermarkets, food distributors, and refrigerated warehouses
Document date	July 15, 2026

Important implementation truth

VITAE currently has a functional API backend prototype, but it does not have a persistent production database. Data is held in Python dictionaries and returns to seeded demo records whenever the server restarts.

. Contents

. Contents	2
1. Executive Summary	3
2. The Product Story	4
3. Application Architecture and Data Sources	5
4. Shipment Lifecycle and Risk Model	6
5. Admin Experience - Platform Control Center	7
6. Organization User Experience - Shipment Operations Center	8
7. Driver Experience - Mobile Delivery App	10
8. Support Agent Experience - Resolution Workspace	11
9. End-to-End Workflow Stories	13
10. Information Access and Permission Matrix	14
11. Backend Technical Guide	15
12. Frontend Structure and Shared Design System	16
13. Validation and Error Handling	17
14. Current Limitations and Production Roadmap	18
15. Glossary and Repository Reference	19

1. Executive Summary

VITAE turns cold-chain telemetry into role-specific decisions before products are damaged.

VITAE is designed for organizations that move or receive temperature-sensitive goods. It is intentionally broader than healthcare: the same shipment, sensor, route, temperature range, and incident concepts apply to medicine, laboratory materials, dairy, frozen food, meat, and fresh produce.

The platform is one application with one authentication system and one visual identity, but four genuinely different working experiences. Admin controls the platform, Organization Users operate shipments, Drivers execute deliveries from a mobile-first interface, and Support Agents resolve cases from a ticket workspace.

The central promise is simple: **know what is moving, know whether it is safe, understand why it is at risk, and record the action taken.**

Current maturity

The role workflows and API mutations are functional and have been verified end to end. Persistence, production identity, IoT authentication, and a deployed ML service remain future infrastructure work.

Core capabilities

- Create a shipment with product, route, schedule, Driver, vehicle or container, sensor, and storage limits.
- Show only the information appropriate to the authenticated role.
- Accept a delivery request, save a mandatory pre-trip checklist, and start a trip.
- Process telemetry, calculate explainable risk, create alerts, and update a shipment timeline.
- Let Organization Users and Drivers request Support while keeping internal Support notes private.
- Complete a delivery, record receiver evidence, verify the result, and retain a final report.

What VITAE is not yet

- It is not yet backed by PostgreSQL, DynamoDB, or another durable database in the default local setup.
- It does not yet use production-grade identity, refresh tokens, password hashing, or a managed authentication provider.
- It does not yet receive authenticated telemetry directly from physical IoT devices.
- The ML integration is an optional external URL and is not configured by default.

2. The Product Story

One shipment creates four different stories because each role has a different responsibility.

Imagine a refrigerated shipment of fresh food leaving a distributor for a supermarket or hospital receiving facility. The Organization User defines the product and safe range, selects an origin and destination, schedules the trip, and assigns accountable resources. The Driver receives only that assigned request, accepts it, confirms the container and cooling checks, and begins the route.

During transit, a sensor reading reports temperature, battery, GPS, and time. VITAE compares the reading with the shipment storage limits. A low-risk condition is labeled Safe, a caution condition is Monitor, a significant excursion is At Risk, and the most urgent combination is Critical. The system records why, creates appropriate alerts, and presents a different amount of detail to each role.

The Organization User sees the operational problem and can notify the Driver or escalate. The Driver receives a short instruction that can be understood on the road. Admin sees the incident in platform context. Support sees shipment diagnostics only when resolving permitted cases; risk alone does not silently create a Support ticket.

At arrival, the Driver records the receiver and notes. The Organization User reviews history and violations, then accepts, flags, or rejects the delivery. That decision becomes the final report and closes the operational story.

Shared domain concepts

Concept	Meaning in VITAE
Organization	The accountable business or institution. It owns facilities, Drivers, resources, shipments, alerts, and tickets.
Facility	A named origin or destination belonging to an Organization.
Product	The goods being transported, including category, quantity, sensitivity, value estimate, and instructions.
Shipment	The central record joining product, route, resources, telemetry, risk, timeline, and delivery evidence.
Driver	A person allowed to see and act only on assigned deliveries.
Vehicle / Container	The refrigerated transport resource assigned to the shipment.
Sensor	The telemetry source associated with temperature, battery, connection, GPS, and last-seen time.
Storage requirements	The minimum and maximum allowed temperature and handling expectations.
Alert	An actionable exception created from a risk condition or operational event.
Ticket	A support conversation with public messages, private internal notes, priority, status, and resolution.

3. Application Architecture and Data Sources

The current repository is a lightweight full-stack prototype. The browser loads role modules written in vanilla JavaScript. Those modules call a Python HTTP server. The server validates the authenticated role, performs business logic, and reads or mutates centralized in-memory dictionaries.

Layer	Technology / behavior	Source file
Browser UI	HTML, CSS, and vanilla JavaScript	index.html, app.js, core/auth.js, core/ui.js, roles/*.js
API layer	Python BaseHTTPRequestHandler / HTTPServer	backend/app.py
Business and storage layer	Functions plus in-memory dictionaries	backend/storage.py
Telemetry processing	Validation, normalization, risk, alerts	sensor_processor.py, risk_rules.py, alerts.py
Live shipment service	Optional DynamoDB scan with local fallback	aws_shipment_service.py
External facility lookup	OpenStreetMap Overpass API	hospital_lookup.py
Optional AI prediction	HTTP call when ML_API_URL is configured	ml_client.py

Where the displayed information comes from

Information	Current store	How it is populated
Organizations	ORGANIZATIONS	Seeded and mutated in storage.py through Admin endpoints
Users and role assignments	USERS	Seeded demo accounts plus Admin-created users
Shipments and timelines	SHIPMENTS	Seeded records plus Organization or Admin shipment APIs
Facilities	FACILITIES	Organization-scoped seeded facility records
Drivers	DRIVERS	Organization-scoped Driver records and workflow status changes
Vehicles	VEHICLES	Organization-scoped transport resources
Sensors	SENSORS	Admin registration, assignment, and latest telemetry updates
Alerts	ORGANIZATION_ALERTS and shipment alerts	Seeded alerts plus sensor-generated exceptions
Support cases	SUPPORT_TICKETS	Organization, Driver, escalation, and Admin-origin requests
Driver incidents	DRIVER_INCIDENTS	Created when a Driver reports an operational problem
Platform preferences	PLATFORM_SETTINGS	Admin-controlled in-memory settings

Persistence warning

These stores are Python process memory, not a real database. API operations are genuinely saved for the lifetime of the server process, but a restart reconstructs the seeded state.

Authentication and routing

A login request checks the USERS dictionary and returns a demo bearer token plus a sanitized profile. The frontend stores the session locally and routes the account to /admin, /organization, /driver, or /support. Every protected API repeats the role check on the server. Frontend route checks improve usability but are not the security boundary.

4. Shipment Lifecycle and Risk Model

State	Meaning	Next system behavior
Planned / Assigned	Organization created the shipment and selected accountable resources.	Driver can see the delivery request.
Accepted	Driver accepted responsibility for the request.	Pickup checklist becomes the next required step.
In Transit	All five pre-trip confirmations were saved.	Organization, Admin, and Support diagnostics show an active trip.
At Risk	Telemetry crossed a safety threshold.	Risk, alerts, recommended actions, and timeline events are updated.
Awaiting Verification	Driver recorded arrival, receiver, and notes.	Organization must accept, flag, or reject.
Delivered / Flagged / Rejected	Organization recorded the final decision.	Final report and complete timeline remain available.

Risk classification

Display class	Internal level	Meaning	Operational response
Safe	low	Conditions are within configured limits.	Continue routine monitoring.
Monitor	medium	A caution condition such as low battery requires observation.	Check the affected resource and watch the next reading.
At Risk	high	Temperature is outside the configured range or another significant condition exists.	Protect the cold chain and coordinate immediately.
Critical	critical	Multiple urgent conditions or a very high score exist.	Immediate intervention and escalation are appropriate.

The rule-based score currently adds 60 points for a temperature excursion, 15 points for low battery, and 30 points for critically low battery. A score of 80 or more is Critical, 50 or more is At Risk, 20 or more is Monitor, and lower values are Safe. Reasons are retained so the UI can explain why the shipment is at risk.

Sensor input and consequences

- The demonstration sensor endpoint requires shipment ID, numeric temperature, battery from 0 to 100, valid GPS coordinates, and a valid timestamp.
- A valid reading updates shipment telemetry, sensor battery and last-seen state, risk level, risk classification, and timeline.
- High or critical readings create Organization alerts and simplified Driver alerts.
- Admin receives the platform-level incident view through its aggregated alert records.
- Support is not automatically added. A ticket appears only after an Organization or Driver request, or an explicit escalation.

5. Admin Experience - Platform Control Center

Purpose

Give a trusted platform operator system-wide control without turning operational shipment work into the Admin's daily job.

The Admin begins with platform health: active shipments, risk, critical incidents, offline sensors, and open tickets. From there, the Admin enters dedicated management workspaces. The experience is desktop-oriented and emphasizes oversight, account governance, device health, and escalation rather than delivery execution.

Information scope

- All Organizations, platform users, shipments, devices, alerts, support tickets, reports, and platform settings.
- Cross-organization aggregates and platform-wide operational totals.
- Shipment diagnostics and routes, but not Driver-only completion actions.
- Ticket assignment and priority, but not private information unrelated to platform operations.

Explicit boundaries

- Admin does not execute a Driver checklist or complete a delivery.
- Admin does not replace Organization Users as the owner of shipment creation and delivery verification.
- Sensitive credentials and raw passwords are not returned in dashboard data.

Section	Features	Information visible	Primary action
Dashboard	Platform health metrics, management tiles, and urgent attention queue.	Aggregated organizations, shipments, sensor health, alerts, and tickets.	Open the filtered management workspace.
Organizations	Search and filter accounts; add, view, edit, suspend, activate, and open related users or shipments.	Organization contact, type, address, status, creation date, and active shipment count.	Create and govern Organization accounts.
Users	Search users; add accounts; edit role and Organization assignment; activate or deactivate.	Sanitized identity, username, email, role, Organization, account status, and activity time.	Control access and role placement.
Shipments	Filter platform shipments; view details and routes.	Organization, product, Driver, route, condition, temperature, risk, status, and ETA.	Read-only operational monitoring.
Devices	Register sensors; view diagnostics; edit assignments; activate or deactivate.	Sensor type, Organization, assignment, connection, battery, last reading, and device state.	Manage telemetry inventory.
Alerts	Search and filter incidents; view, acknowledge, escalate, or resolve.	Severity, shipment, Organization, explanation, recommended action, and Driver response.	Platform-level incident governance.
Support Tickets	View, assign, prioritize, escalate, or close cases.	Ticket source, reporting user, linked shipment, priority, assigned agent, status, and timestamps.	Oversee Support workload.
Reports	Review shipment, device, support, protection, and network composition totals.	Aggregates calculated from current in-memory records.	Platform reporting.
Settings	Configure display name and warning, battery, and notification preferences.	PLATFORM_SETTINGS values.	Save safe MVP preferences.

6. Organization User Experience - Shipment Operations Center

Purpose

Operate one Organization's shipments from creation through final delivery verification.

The Organization User starts from a concise operations cockpit. The main story is ownership: define the shipment, assign accountable resources, watch the trip, respond to risk, coordinate with the Driver, and verify the final handoff. Every backend query derives the Organization ID from the authenticated account rather than trusting an Organization ID submitted by the browser.

Information scope

- Only the authenticated Organization's shipments, facilities, Drivers, vehicles, sensors, alerts, tickets, and reports.
- Product quantity, financial estimate, handling instructions, route, telemetry, risk explanation, and delivery evidence for owned shipments.
- Public Support messages for owned tickets; internal Support notes are removed server-side.

Explicit boundaries

- Cannot view another Organization's private shipments by changing a URL or request body.
- Cannot assign a Driver, facility, vehicle, or sensor owned by another Organization.
- Cannot select an offline sensor or unavailable Driver for a new shipment.
- Cannot manage platform-wide users, Organizations, or devices.

Section	Features	Information visible
Dashboard	Essential metrics, up to three live shipments, prioritized action center, next arrival, Driver capacity, and latest timeline update.	Owned active shipments, risks, arrivals, Driver availability, alerts, and recent events.
Create Shipment	Four-step Product, Delivery, Assignment, and Review workflow with inline validation and duplicate submission protection.	Owned facilities, eligible Drivers, owned vehicles, online sensors, and entered product data.
Shipments	Search, filter, sort, inspect condition, route, risk analysis, timeline, assignment, temperature history, verification, and final report.	All fields for owned shipments, including financial estimate and delivery evidence.
Live Tracking	Five-second live refresh, selected route, current location, temperature, range, battery, sensor state, Driver, and update time.	Active owned shipments from DynamoDB when configured, otherwise local SHIPMENTS fallback.
Alerts	Review explanation and recommendation; acknowledge, notify Driver, record action, escalate to Support, resolve.	Owned alerts and the latest recorded Driver response.
Drivers	Search availability; view assignment and delivery history; update availability; assign eligible planned shipments.	Drivers belonging to the Organization only.
Reports	Shipment totals, safe and risk totals, on-time result, protected value estimate, Driver summary, outcomes, and alert summary.	Calculated from owned shipments and alerts.
Support	Create a ticket, link a shipment or alert, choose urgency, view history, and send public replies.	Owned tickets and public conversation only.

Create Shipment field story

Step	Information collected	Backend control
1. Product	Name, category, quantity, unit, estimated value, minimum and maximum temperature, expiration, sensitivity, and handling notes.	Storage limits must be numeric and minimum must be below maximum.
2. Delivery	Owned origin and destination facilities, departure, expected arrival, and instructions.	Origin and destination must differ; arrival must be after departure.
3. Assignment	An eligible Organization Driver, owned vehicle or container, and non-offline owned sensor.	Every selected record is checked against the authenticated Organization.
4. Review	Named route and resources, schedule, temperature range, and estimated value.	One create action sends the complete draft and disables repeat submission while saving.

What is saved

A successful request writes one shipment record, marks the Driver assigned, links the sensor to the shipment, records Shipment Created in the timeline, and returns the created object. The Organization dashboard reloads from the API and the Driver dashboard can immediately discover the request through the shared shipment store.

Duplicate and failure behavior

- A submissionId makes a retried review submission idempotent: the existing shipment is returned instead of creating a second record.
- Invalid resources, offline sensors, unavailable Drivers, impossible schedules, unsupported categories, and invalid numeric values return backend errors.
- The wizard retains the draft and shows the backend error inline when creation fails.
- Success feedback appears only after the API confirms the record was saved.

7. Driver Experience - Mobile Delivery App

Purpose

Help a Driver understand the next required action in seconds while exposing only assigned deliveries.

The Driver experience follows a delivery-app pattern. New requests are the starting point. Acceptance changes the request into a committed delivery. A required pickup checklist gates the trip. During transit, destination, ETA, temperature, range, instruction, route, alerts, incident reporting, and arrival confirmation stay within reach. Completed deliveries become a personal trip record.

Information scope

- Only shipments whose driverId matches the authenticated Driver.
- Operational product category, pickup, destination, schedule, safe range, current condition, Organization contact, and handling instructions.
- Driver-specific alerts, incidents, trip history, and Driver-created Support conversations.
- No estimated financial value or unrelated Organization information.

Explicit boundaries

- Cannot discover or open an unassigned shipment through direct API requests.
- Cannot create shipments, reassign Drivers, manage devices, or verify delivery outcomes.
- Cannot complete a trip twice or complete a delivery that is not active.
- Cannot see Support internal notes.

Section	Features	Information visible
Home	Prominent current or next delivery, new delivery requests, active trip status, and recent trip history.	Assigned requests, accepted work, active deliveries, and completed records.
My Deliveries	Request, active, and history tabs; compact cards; delivery details; acceptance and checklist workflow.	Shipment ID, product category, route, deadline, status, safe range, condition, contact, and instructions.
Active Trip	Destination, route summary, ETA, temperature, required range, main instruction, route map, incident form, and arrival workflow.	One selected active assigned shipment.
Alerts	Short actionable warning, one main action, and secondary options for continuing problems, Organization contact, Support, or sensor issue.	Only unresolved alerts for assigned shipments.
Support	Create a short help request, link an assigned shipment, choose issue type, and read public responses.	Driver's own Support tickets and public messages.
Profile	Identity, Organization, vehicle, availability, and completed trip count.	Authenticated Driver record and assigned delivery history.

Mandatory pre-trip checklist

- Shipment collected
- Container closed
- Sensor connected
- Cooling active
- Vehicle ready

The backend rejects the Start Delivery request unless every value is explicitly true. On success it saves who confirmed the checks, the confirmation time, the in-transit status, the Driver on-route state, and a shipment timeline event.

8. Support Agent Experience - Resolution Workspace

Purpose

Resolve technical and shipment-related cases with sufficient diagnostics but without operational control over shipments.

Support begins with a unified intake queue rather than a platform dashboard. Requests show their source - Admin, Organization, or Driver - and are prioritized beside a read-only live-trip monitor. Opening a ticket creates a two-column resolution workspace: public conversation and case controls on the left, shipment and customer diagnostics on the right.

Information scope

- Permitted Support tickets, their public conversation, Support-only internal notes, priority, assignment, status, and resolution summary.
- Read-only diagnostic shipment context: status, location, temperature, required range, sensor connection, battery, alerts, and actions already taken.
- Limited Organization contact and ticket history needed for assistance.
- All ongoing trips in the current Support monitoring scope, without financial or shipment-management controls.

Explicit boundaries

- Cannot create, assign, start, complete, verify, or otherwise modify shipments.
- Cannot manage Organizations, platform users, or devices.
- Internal notes never appear in Organization or Driver API responses.
- Escalation goes to Admin; Support does not gain Admin permissions.

Section	Features	Information visible
Dashboard	Workload summary, source counts, unified incoming queue, and live trips.	Tickets from Admin, Organizations, and Drivers plus read-only active trip diagnostics.
All Tickets	Search, priority/status/Organization/agent filters, sorting, pagination, and row menu.	Full permitted ticket list.
Critical Tickets	Focused unresolved critical queue with the same resolution workflow.	Critical-priority permitted tickets.
Shipment Lookup	Search by shipment, Organization, Driver, or linked ticket.	Read-only diagnostic records without financial or assignment controls.
Organizations	Search limited Organization profiles and review open/ticket history.	Name, type, main contact, region, and support history.
Knowledge Base	Search and expand guidance for sensor, battery, missing readings, GPS, cooling, login, and lookup issues.	Static troubleshooting guidance from the backend response.
Profile	Agent identity, assigned open-ticket count, access description, and logout.	Authenticated Support profile and assigned workload.

Ticket detail and resolution story

- Reply publicly or request more information.
- Add a visually distinct internal note that remains private.
- Change priority and status.
- Escalate to Admin.
- Resolve only with a resolution summary.
- Retain public conversation and ticket history after resolution.

Visibility by content type

Content	Who can see it	Purpose
Problem description	Support, requester, and Admin oversight	Initial public case context.
Public reply	Support and requester	Used for instructions, questions, and confirmed outcomes.
Internal note	Support only	Diagnostic reasoning and handoff notes; removed from requester API responses.
Shipment diagnostics	Support read-only	Status, temperature, range, sensor, battery, location, alerts, and actions.
Resolution summary	Support, requester, and Admin oversight	Required before the ticket can become Resolved.

Ticket status path

A case begins as New, normally moves to In Progress, may wait for the user, may be Escalated to Admin, and becomes Resolved only with a resolution summary. A public requester reply can return a waiting case to In Progress. The current MVP deliberately prevents reopening a resolved ticket.

Support safety boundary

Shipment context is diagnostic only. No Support form or endpoint can create a shipment, change assignment, start a trip, complete a delivery, or verify the result.

9. End-to-End Workflow Stories

Workflow	Initiator	Main API	Backend effect	Cross-role result
1. Create shipment	Organization User	POST /api/organization/shipments	Validates ownership and resources, writes SHIPMENTS, assigns Driver and sensor.	Organization list and assigned Driver request both update.
2. Start delivery	Driver	PATCH accept, then PATCH start	Checks Driver ownership and all five confirmations, updates status and timeline.	Organization sees in-transit status.
3. Risk and alert	Admin demo sensor source	POST /api/sensor-data	Validates telemetry, stores reading, calculates risk, creates alerts, updates timeline.	Organization, Driver, and Admin see role-specific incident information.
4. Support request	Organization or Driver	POST ticket endpoint	Creates linked SUPPORT_TICKETS record. Support replies and notes through separate endpoints.	Public replies return to requester; internal notes remain private.
5. Complete delivery	Driver then Organization	PATCH complete, then PATCH verification	Stores receiver and notes, prevents duplicate completion, records final decision and timeline.	Final report is available in the owned shipment detail.

Workflow integrity principles

- The browser never treats a button click as success until the API returns success.
- Mutation endpoints validate role, ownership, current state, required fields, and allowed transitions.
- Timelines provide a shared audit narrative across creation, trip start, risk, alert response, Support request, completion, and verification.
- Organization IDs submitted by the browser are not authoritative for Organization-scoped operations.
- Duplicate shipment creation is controlled by submissionId; duplicate delivery completion is rejected by shipment state.

10. Information Access and Permission Matrix

Information / action	Admin	Organization	Driver	Support
All Organizations	Manage	No	No	Limited support view
Platform users	Manage	No	No	No
All shipments	View	No - owned only	No - assigned only	Diagnostic view
Create shipment	Legacy/admin API only	Yes	No	No
Assign Driver	Oversight only	Yes - own Drivers	No	No
Start / complete trip	No	No	Yes - assigned only	No
Verify delivery	No	Yes - owned only	No	No
Alerts	All	Owned	Assigned shipment alerts	Ticket-related context
Tickets	All oversight	Owned Organization tickets	Own Driver tickets	Permitted resolution queue
Internal notes	Ticket oversight only	Never	Never	Create and view
Financial estimate	Platform shipment view	Owned shipment view	Never	Never
Device management	Yes	Owned selection/read state	No	No

Server-side enforcement

The API uses `require_admin_user`, `require_organization_user`, `require_driver_user`, and `require_support_user` before business functions run. Organization-owned records pass through an ownership check. Driver delivery access compares the authenticated `driverId` with the shipment `driverId`. Suspended Organizations and inactive accounts are denied even if an old browser route is still open.

Privacy guarantee

Support internal notes are removed while constructing Organization and Driver ticket records. This is a backend data-shaping rule, not only a hidden frontend element.

11. Backend Technical Guide

File	Responsibility
<code>backend/app.py</code>	HTTP routing, JSON parsing, response codes, authentication gates, role gates, and static frontend serving.
<code>backend/storage.py</code>	Seed data, record dictionaries, data shaping, permissions helpers, validation, workflow mutations, reports, and timelines.
<code>backend/sensor_processor.py</code>	Required telemetry validation, normalization, and orchestration of risk, lookup, ML, alerts, and storage.
<code>backend/risk_rules.py</code>	Explainable temperature and battery scoring.
<code>backend/alerts.py</code>	Builds rule-based, ML, and battery alert messages.
<code>backend/aws_shipment_service.py</code>	Reads live shipments from DynamoDB when <code>AWS_REGION</code> and <code>SHIPMENTS_TABLE</code> are set; otherwise reads local <code>SHIPMENTS</code> .
<code>backend/hospital_lookup.py</code>	Optional OpenStreetMap Overpass lookup for a nearby compatible hospital.
<code>backend/ml_client.py</code>	Calls an external model when <code>ML_API_URL</code> is set; otherwise returns not configured.

Main role dashboard responses

Endpoint	Response scope	Backend function
<code>GET /api/admin/dashboard</code>	All platform aggregates and management collections	<code>get_admin_foundation_dashboard_data</code>
<code>GET /api/organization/dashboard</code>	Owned operations bundle	<code>get_organization_foundation_dashboard_data</code>
<code>GET /api/driver/dashboard</code>	Assigned deliveries, alerts, tickets, and incidents	<code>get_driver_dashboard_data</code>
<code>GET /api/support/dashboard</code>	Ticket workload plus diagnostic context	<code>get_support_foundation_dashboard_data</code>
<code>GET /api/shipments/live</code>	Role-scoped active shipment rows	<code>get_live_shipments_for_user</code>

Mutation endpoint families

- Admin: create/update Organizations, users, sensors, alerts, tickets, and settings.
- Organization: create shipments and tickets; assign Drivers; update alerts and availability; verify delivery.
- Driver: accept, start, and complete assigned shipments; respond to alerts; report incidents; create Support requests.
- Support: reply, add internal notes, change ticket status or priority, escalate, and resolve.
- Telemetry demonstration: Admin-authorized `POST /api/sensor-data`.

12. Frontend Structure and Shared Design System

File	Responsibility
index.html	Login, denied view, role view mounts, and ordered script loading.
app.js	Boot, route selection, role event delegation, dashboard loading, live polling, maps, loading state, retry state, and feedback.
core/auth.js	Session storage, login, API wrapper, network errors, and expired-session event.
core/ui.js	Shared shell, page headers, badges, empty states, escaping, and formatting.
roles/admin.js	Admin control center and management actions.
roles/organization.js	Organization operations, shipment wizard, alerts, verification, reports, and Support.
roles/driver.js	Mobile request, trip, alert, incident, completion, and history experience.
roles/support.js	Ticket resolution, diagnostics, lookup, organizations, knowledge base, and profile.
services/shipmentApi.js	Five-second role-authenticated live shipment request.
styles.css	Shared VITAE identity plus role-specific layouts and responsive rules.

Shared versus distinct

All roles share typography, color tokens, badges, forms, error language, dialogs, authentication, and reusable shipment concepts. Their information hierarchy is intentionally different: Admin is a broad control center, Organization is an operational cockpit, Driver is a focused mobile flow, and Support is a desktop resolution queue.

Loading and error behavior

- Workspace loading state appears while the role dashboard API is requested.
- A failed workspace request produces a page-level explanation and Retry action.
- Network failures produce an understandable server-connection message.
- Expired sessions clear local state and return the user to sign-in with an explanation.
- Operation failures appear as page-level feedback as well as a short status toast.
- Empty data collections show explicit empty states instead of blank panels.

13. Validation and Error Handling

Case	Control	User-visible result
Invalid form	Browser required fields plus backend required-field, numeric, range, ownership, and state validation.	400 with a useful error message.
Missing reading	Sensor payload requires shipment, temperature, battery, GPS, and timestamp.	400 naming the missing field.
Offline sensor	Shipment creation rejects an offline selected sensor.	400: selected sensor is offline.
Low battery	Risk score and alert generation handle low and critical battery.	Monitor or Critical plus recommended action.
Permission denied	Role guard runs before operation.	403 with role-specific requirement.
Expired session	Invalid or inactive token is rejected; frontend clears session.	401 and sign-in explanation.
Invalid shipment ID	Owned or assigned lookup fails without leaking whether another user owns it.	404 Shipment or Delivery not found.
Unavailable Organization	Missing or suspended Organization fails server-side role validation.	403 Organization or Driver access required.
Unavailable Driver	Shipment creation checks Driver ownership and availability.	400 selected Driver is not available.
Duplicate completion	Completion requires an active status.	400 only an active delivery can be completed.
Failed ticket creation	Category, urgency, description, and optional linked ownership are validated.	400 or 404 without a fake success state.
Network failure	Frontend API wrapper catches fetch failure.	Page-level message with retry path.

14. Current Limitations and Production Roadmap

Current backend classification

A functional in-memory prototype backend. It is appropriate for demonstrations and workflow validation, but not for production custody of cold-chain records.

Phase	Production work
1. Persistent data	Replace dictionaries with PostgreSQL or DynamoDB repositories, migrations, indexes, transactions, backups, and audit retention.
2. Production identity	Use managed authentication, hashed passwords, secure HTTP-only cookies or short-lived tokens, refresh and revocation, MFA, and recovery.
3. IoT ingestion	Authenticate devices through AWS IoT Core or equivalent, validate device-to-sensor assignment, queue events, and handle replay/idempotency.
4. Durable files	Store incident images and proof-of-delivery files in protected object storage with access policies.
5. ML operations	Deploy and version the spoilage model, monitor quality, retain prediction explanations, and define fallback behavior.
6. Notifications	Add email, SMS, push, or messaging integrations with delivery tracking and escalation rules.
7. Audit and compliance	Immutable event log, retention policies, data residency, consent, access reviews, and industry-specific controls.
8. Testing and delivery	Formal unit, integration, browser, accessibility, security, load, and migration tests with CI/CD.

Data migration recommendation

Preserve the current service functions and role-shaped response contracts, but move record access behind repository interfaces. This allows the frontend to remain stable while SHIPMENTS, USERS, SUPPORT_TICKETS, and the other dictionaries are replaced by durable tables or collections. Timeline events should become append-only records rather than nested mutable arrays.

15. Glossary and Repository Reference

Term	Definition
Cold chain	The controlled-temperature handling path from origin to destination.
Telemetry	Sensor measurements such as temperature, battery, GPS, and timestamp.
Risk explanation	The rule reasons retained with a risk score so users know why action is needed.
Timeline	Chronological shipment events recording decisions and workflow transitions.
Public message	A Support conversation message visible to the requester.
Internal note	A Support-only diagnostic note removed from requester API responses.
Verification	The Organization User's final accept, flag, or reject decision after Driver completion.
Local fallback	The live service uses in-memory SHIPMENTS when DynamoDB environment variables are absent.
MVP	Minimum viable product: a working prototype used to validate product behavior.

Primary repository paths

- SD/frontend - role interfaces, shared UI, authentication wrapper, live shipment service, and styling.
- SD/backend - HTTP API, storage and workflow logic, telemetry, alerts, risk, AWS fallback, lookup, and ML client.
- SD.md - local run command and endpoint overview.

Summary

VITAE already demonstrates the full product story across four coordinated but distinct interfaces. Its next major step is not another dashboard redesign; it is replacing process memory with durable infrastructure while preserving the verified role boundaries and workflow contracts.